

The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style. Do not include any additional commentary, warnings or suggestions outside these comments in your response.

```
class TaskQueue:
    # init
    def __init__(self, max_size=100):
        self.queue = []          # list
        self.max = max_size      # cap

    # add task
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False         # full
        self.queue.append(task)
        return True

    # remove task
    def dequeue(self):
        if not self.queue:
            return None          # empty
        return self.queue.pop(0)

    # check empty
    def is_empty(self):
        return len(self.queue) == 0

    # size
    def size(self):
        return len(self.queue)

    # peek
    def peek(self):
        if self.queue:
            return self.queue[0]
        return None
```

```

class RateLimiter:
    # setup
    def __init__(self, limit, window):
        self.limit = limit        # max calls
        self.window = window      # seconds
        self.calls = []           # timestamps

    # check
    def allow(self, timestamp):
        # remove old
        self.calls = [t for t in self.calls if
timestamp - t < self.window]
        if len(self.calls) < self.limit:
            self.calls.append(timestamp)
            return True
        return False              # denied

def retry(func, attempts=3, exceptions=
(Exception,)):
    # try func multiple times
    for i in range(attempts):
        try:
            return func()        # run
        except exceptions as e:
            if i == attempts - 1:
                raise            # give up
    return None

```

&lt;/&gt; Python



▶ Run

```

class TaskQueue:
    # Initialize a bounded first-in, first-out (FIFO) task queue.
    # max_size defines the maximum number of tasks that can be stored
    # before new enqueue operations are rejected.
    def __init__(self, max_size=100):
        self.queue = []          # Internal storage for queued tasks in arrival order.
        self.max = max_size       # Maximum allowed queue length.

    # Add a task to the end of the queue.
    # Returns:
    #   True - task was successfully added.
    #   False - queue is already at capacity.
    def enqueue(self, task):

```

```

    if len(self.queue) >= self.max:
        return False      # Reject new tasks when capacity is reached.
    self.queue.append(task)
    return True

# Remove and return the task at the front of the queue.
# Returns:
#   The oldest queued task if one exists.
#   None if the queue is empty.
def dequeue(self):
    if not self.queue:
        return None      # No tasks available to process.
    return self.queue.pop(0)

# Determine whether the queue currently contains any tasks.
# Returns:
#   True if the queue is empty, otherwise False.
def is_empty(self):
    return len(self.queue) == 0

# Return the current number of tasks waiting in the queue.
def size(self):
    return len(self.queue)

# Retrieve the next task scheduled for processing without removing it.
# Returns:
#   The task at the front of the queue.
#   None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Initialize a simple sliding-window rate limiter.
    #
    # Parameters:
    #   limit - maximum number of allowed requests within the window.
    #   window - rolling time window length in seconds.
    #
    # The limiter stores timestamps for recent successful requests and
    # evaluates each new request against the configured threshold.
    def __init__(self, limit, window):
        self.limit = limit      # Maximum allowed requests per window.
        self.window = window    # Size of the rolling time window in seconds.
        self.calls = []         # Timestamps of accepted requests.

    # Determine whether a request at the given timestamp should be allowed.
    #
    # Expired timestamps that fall outside the current window are removed
    # before evaluating the request count.

```

```

#
# Returns:
#   True - request is permitted and recorded.
#   False - rate limit has been exceeded.
def allow(self, timestamp):
    # Retain only timestamps that still fall within the active window.
    self.calls = [t for t in self.calls if timestamp - t < self.window]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False          # Request denied because the limit was reached.

def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable and automatically retry on specified exceptions.
    #
    # Parameters:
    #   func      - callable to execute.
    #   attempts  - total number of execution attempts.
    #   exceptions - tuple of exception types that should trigger a retry.
    #
    # Returns:
    #   The result produced by func() if any attempt succeeds.
    #
    # Raises:
    #   The last caught exception after all retry attempts are exhausted.
    for i in range(attempts):
        try:
            return func()          # Attempt execution.
        except exceptions as e:
            if i == attempts - 1:
                raise              # No retries remain; propagate the exception.

    # Defensive fallback. This path is normally unreachable because the
    # function either returns a result or raises an exception.
    return None

```



The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style. Do not include any additional commentary, warnings or suggestions outside these comments in your response.

```
class TaskQueue:
    # Initialize a bounded first-in, first-out (FIFO) task queue.
    # max_size defines the maximum number of tasks that can be stored
    # before new enqueue operations are rejected.
    def __init__(self, max_size=100):
        self.queue = []          # Internal storage for queued tasks in arrival order.
        self.max = max_size      # Maximum allowed queue length.

    # Add a task to the end of the queue.
    # Returns:
    #   True - task was successfully added.
    #   False - queue is already at capacity.
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False          # Reject new tasks when capacity is reached.
        self.queue.append(task)
        return True

    # Remove and return the task at the front of the queue.
    # Returns:
    #   The oldest queued task if one exists.
    #   None if the queue is empty.
    def dequeue(self):
        if not self.queue:
            return None           # No tasks available to process.
        return self.queue.pop(0)

    # Determine whether the queue currently contains any tasks.
    # Returns:
    #   True if the queue is empty, otherwise False.
    def is_empty(self):
        return len(self.queue) == 0
```

```
# Return the current number of tasks
waiting in the queue.
def size(self):
    return len(self.queue)

# Retrieve the next task scheduled for
processing without removing it.
# Returns:
#   The task at the front of the queue.
#   None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]
    return None

class RateLimiter:
    # Initialize a simple sliding-window rate
    limiter.
    #
    # Parameters:
    #   limit - maximum number of allowed
    requests within the window.
    #   window - rolling time window length in
    seconds.
    #
    # The limiter stores timestamps for recent
    successful requests and
    # evaluates each new request against the
    configured threshold.
    def __init__(self, limit, window):
        self.limit = limit      # Maximum
allowed requests per window.
        self.window = window   # Size of the
rolling time window in seconds.
        self.calls = []        # Timestamps of
accepted requests.

    # Determine whether a request at the given
    timestamp should be allowed.
    #
    # Expired timestamps that fall outside the
    current window are removed
```

```
# before evaluating the request count.
#
# Returns:
#   True - request is permitted and
recorded.
#   False - rate limit has been exceeded.
def allow(self, timestamp):
    # Retain only timestamps that still
fall within the active window.
    self.calls = [t for t in self.calls if
timestamp - t < self.window]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False # Request
denied because the limit was reached.
```

```
def retry(func, attempts=3, exceptions=
(Exception,)):
    # Execute a callable and automatically
retry on specified exceptions.
    #
    # Parameters:
    #   func - callable to execute.
    #   attempts - total number of execution
attempts.
    #   exceptions - tuple of exception types
that should trigger a retry.
    #
    # Returns:
    #   The result produced by func() if any
attempt succeeds.
    #
    # Raises:
    #   The last caught exception after all
retry attempts are exhausted.
    for i in range(attempts):
        try:
            return func() # Attempt
execution.
```

```

except exceptions as e:
    if i == attempts - 1:
        raise          # No retries
    remain; propagate the exception.

```

```

    # Defensive fallback. This path is normally
    unreachable because the
    # function either returns a result or
    raises an exception.
    return None

```

&lt;/&gt; Python



▶ Run

```

class TaskQueue:
    # Create a bounded first-in, first-out (FIFO) queue for task management.
    #
    # Tasks are stored in the order they are received and processed in the
    # same order. The queue enforces a maximum capacity to prevent unbounded
    # growth and provides simple operations for adding, removing, and
    # inspecting queued work.
    #
    # Args:
    #     max_size (int): Maximum number of tasks that may be stored in the
    #                     queue at one time. Additional enqueue attempts are rejected
    #                     once this limit is reached.
    def __init__(self, max_size=100):
        self.queue = []          # Ordered collection of pending tasks.
        self.max = max_size      # Upper bound on the number of queued tasks.

    # Add a task to the back of the queue.
    #
    # The task is appended only if sufficient capacity remains.
    #
    # Args:
    #     task: Object representing the work item to be queued.
    #
    # Returns:
    #     bool:
    #         True if the task was successfully queued.
    #         False if the queue is already full.
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False          # Capacity reached; reject the new task.

        self.queue.append(task)
        return True

```



```
# Remove and return the next task to be processed.
#
# Because the queue follows FIFO semantics, the oldest queued task is
# always returned first.
#
# Returns:
#     The task at the front of the queue, or None if no tasks are
#     currently waiting.
def dequeue(self):
    if not self.queue:
        return None        # No pending tasks are available.

    return self.queue.pop(0)

# Check whether the queue currently contains any tasks.
#
# Returns:
#     bool: True if the queue is empty, otherwise False.
def is_empty(self):
    return len(self.queue) == 0

# Get the current number of queued tasks.
#
# Returns:
#     int: Total number of tasks waiting in the queue.
def size(self):
    return len(self.queue)

# Inspect the next task scheduled for processing without removing it.
#
# This operation allows callers to view the upcoming task while leaving
# the queue unchanged.
#
# Returns:
#     The task at the front of the queue, or None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]

    return None

class RateLimiter:
    # Implement a fixed-capacity sliding-window rate limiter.
    #
    # The limiter tracks timestamps of accepted requests and determines
    # whether a new request can be allowed based on how many requests have
    # occurred within the configured time window.
    #
    # Args:
    #     limit (int): Maximum number of requests permitted during a single
    #     rolling time window.
```

```

#     window (int | float): Duration of the rolling window in seconds.
def __init__(self, limit, window):
    self.limit = limit      # Maximum requests allowed within the window.
    self.window = window    # Rolling time window length in seconds.
    self.calls = []         # Timestamps for previously accepted requests.

# Evaluate whether a request should be allowed at the specified time.
#
# Requests that fall outside the active window are discarded before the
# current request is evaluated. If the number of remaining requests is
# below the configured limit, the request is accepted and its timestamp
# is recorded.
#
# Args:
#     timestamp (int | float): Time associated with the incoming request.
#
# Returns:
#     bool:
#         True if the request is permitted.
#         False if the rate limit has been reached.
def allow(self, timestamp):
    # Remove request records that are no longer relevant to the current
    # rolling window.
    self.calls = [
        t for t in self.calls
        if timestamp - t < self.window
    ]

    if len(self.calls) < self.limit:
        self.calls.append(timestamp)
        return True

    return False          # Request exceeds the allowed request rate.

def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable with automatic retry behavior.
    #
    # The target function is invoked repeatedly until it succeeds or the
    # maximum number of attempts has been exhausted. Only exceptions whose
    # types are included in the exceptions parameter trigger a retry.
    #
    # Args:
    #     func (callable): Function to execute.
    #     attempts (int): Total number of execution attempts, including the
    #         initial call.
    #     exceptions (tuple[type]): Exception types that should be caught and
    #         retried.
    #
    # Returns:
    #     The value returned by func() if an attempt succeeds.
    #

```

```

# Raises:
#     Any exception not listed in exceptions.
#     The final caught exception if all retry attempts fail.
def retry(func, attempts=3, exceptions=(Exception,)):
    for i in range(attempts):
        try:
            return func() # Execute the target operation.

        except exceptions:
            # Re-raise the exception once the final retry has failed.
            if i == attempts - 1:
                raise

    # Defensive fallback. Under normal execution this statement is never
    # reached because the function either returns a result or raises an
    # exception before the loop completes.
    return None

```



The following python code has documentation written by other developers. Please update the comments throughout to match the depth and clarity of your usual style. Do not include any additional commentary, warnings or suggestions outside these comments in your response.

```

class TaskQueue:
    # Create a bounded first-in, first-out
    # (FIFO) queue for task management.
    #
    # Tasks are stored in the order they are
    # received and processed in the
    # same order. The queue enforces a maximum
    # capacity to prevent unbounded
    # growth and provides simple operations for
    # adding, removing, and
    # inspecting queued work.
    #
    # Args:
    #     max_size (int): Maximum number of
    # tasks that may be stored in the
    # queue at one time. Additional
    # enqueue attempts are rejected
    # once this limit is reached.

```

```

def __init__(self, max_size=100):
    self.queue = []          # Ordered
collection of pending tasks.
    self.max = max_size      # Upper bound
on the number of queued tasks.

    # Add a task to the back of the queue.
    #
    # The task is appended only if sufficient
capacity remains.
    #
    # Args:
    #     task: Object representing the work
item to be queued.
    #
    # Returns:
    #     bool:
    #         True if the task was successfully
queued.
    #         False if the queue is already
full.
    def enqueue(self, task):
        if len(self.queue) >= self.max:
            return False      # Capacity
reached; reject the new task.

        self.queue.append(task)
        return True

    # Remove and return the next task to be
processed.
    #
    # Because the queue follows FIFO semantics,
the oldest queued task is
    # always returned first.
    #
    # Returns:
    #     The task at the front of the queue,
or None if no tasks are
    #     currently waiting.
    def dequeue(self):
        if not self.queue:
            return None       # No pending

```

tasks are available.

```
        return self.queue.pop(0)

    # Check whether the queue currently
    contains any tasks.
    #
    # Returns:
    #     bool: True if the queue is empty,
    otherwise False.
    def is_empty(self):
        return len(self.queue) == 0

    # Get the current number of queued tasks.
    #
    # Returns:
    #     int: Total number of tasks waiting in
    the queue.
    def size(self):
        return len(self.queue)

    # Inspect the next task scheduled for
    processing without removing it.
    #
    # This operation allows callers to view the
    upcoming task while leaving
    # the queue unchanged.
    #
    # Returns:
    #     The task at the front of the queue,
    or None if the queue is empty.
    def peek(self):
        if self.queue:
            return self.queue[0]

        return None

class RateLimiter:
    # Implement a fixed-capacity sliding-window
    rate limiter.
    #
    # The limiter tracks timestamps of accepted
```

```
requests and determines
    # whether a new request can be allowed
based on how many requests have
    # occurred within the configured time
window.
    #
    # Args:
    #     limit (int): Maximum number of
requests permitted during a single
    #         rolling time window.
    #     window (int | float): Duration of the
rolling window in seconds.
    def __init__(self, limit, window):
        self.limit = limit        # Maximum
requests allowed within the window.
        self.window = window     # Rolling time
window length in seconds.
        self.calls = []          # Timestamps
for previously accepted requests.

    # Evaluate whether a request should be
allowed at the specified time.
    #
    # Requests that fall outside the active
window are discarded before the
    # current request is evaluated. If the
number of remaining requests is
    # below the configured limit, the request
is accepted and its timestamp
    # is recorded.
    #
    # Args:
    #     timestamp (int | float): Time
associated with the incoming request.
    #
    # Returns:
    #     bool:
    #         True if the request is permitted.
    #         False if the rate limit has been
reached.
    def allow(self, timestamp):
        # Remove request records that are no
longer relevant to the current
```

```
# rolling window.
self.calls = [
    t for t in self.calls
    if timestamp - t < self.window
]

if len(self.calls) < self.limit:
    self.calls.append(timestamp)
    return True

return False          # Request
exceeds the allowed request rate.

def retry(func, attempts=3, exceptions=
(Exception,)):
    # Execute a callable with automatic retry
    behavior.
    #
    # The target function is invoked repeatedly
    until it succeeds or the
    # maximum number of attempts has been
    exhausted. Only exceptions whose
    # types are included in the exceptions
    parameter trigger a retry.
    #
    # Args:
    #     func (callable): Function to execute.
    #     attempts (int): Total number of
    execution attempts, including the
    #         initial call.
    #     exceptions (tuple[type]): Exception
    types that should be caught and
    #         retried.
    #
    # Returns:
    #     The value returned by func() if an
    attempt succeeds.
    #
    # Raises:
    #     Any exception not listed in
    exceptions.
    #     The final caught exception if all
```

```

retry attempts fail.
    def retry(func, attempts=3, exceptions=
(Exception,)):
        for i in range(attempts):
            try:
                return func()    # Execute the
target operation.

            except exceptions:
                # Re-raise the exception once
the final retry has failed.
                if i == attempts - 1:
                    raise

        # Defensive fallback. Under normal
execution this statement is never
        # reached because the function either
returns a result or raises an
        # exception before the loop completes.
        return None

```

&lt;/&gt; Python



Run

```

class TaskQueue:
    # Manage a bounded first-in, first-out (FIFO) collection of tasks.
    #
    # Tasks are processed in the same order they are added. The queue
    # enforces a configurable capacity limit to prevent uncontrolled growth
    # and provides basic operations for task insertion, retrieval, and
    # inspection.
    #
    # Args:
    #     max_size (int): Maximum number of tasks that may be stored at any
    #         given time. Attempts to add tasks beyond this limit will fail.
    def __init__(self, max_size=100):
        self.queue = []    # Internal FIFO storage for pending tasks.
        self.max = max_size    # Maximum number of tasks allowed in memory.

    # Insert a task at the end of the queue.
    #
    # The task becomes the most recently queued item and will not be
    # processed until all previously queued tasks have been removed.
    #
    # Args:
    #     task: The task object to enqueue.

```



```
#
# Returns:
#     bool:
#         True if the task was successfully added.
#         False if the queue has already reached its capacity limit.
def enqueue(self, task):
    if len(self.queue) >= self.max:
        return False        # Reject the task when no capacity remains.

    self.queue.append(task)
    return True

# Remove and return the task at the front of the queue.
#
# This operation preserves FIFO behavior by always returning the oldest
# pending task.
#
# Returns:
#     The next task scheduled for processing, or None if the queue
#     contains no tasks.
def dequeue(self):
    if not self.queue:
        return None        # No queued work is available.

    return self.queue.pop(0)

# Determine whether the queue currently contains any pending tasks.
#
# Returns:
#     bool: True when the queue is empty; otherwise False.
def is_empty(self):
    return len(self.queue) == 0

# Return the number of tasks currently waiting in the queue.
#
# Returns:
#     int: Current queue length.
def size(self):
    return len(self.queue)

# Retrieve the next task to be processed without removing it.
#
# Unlike dequeue(), this method leaves the queue unchanged and is useful
# when callers need to inspect the upcoming task before processing.
#
# Returns:
#     The task at the front of the queue, or None if the queue is empty.
def peek(self):
    if self.queue:
        return self.queue[0]

    return None
```

```

class RateLimiter:
    # Enforce a request limit using a sliding time-window algorithm.
    #
    # The limiter records timestamps for accepted requests and evaluates each
    # new request against the number of requests that occurred within the
    # configured rolling window.
    #
    # Args:
    #     limit (int): Maximum number of requests permitted within a single
    #         active window.
    #     window (int | float): Window duration in seconds.
    def __init__(self, limit, window):
        self.limit = limit        # Maximum requests allowed per window.
        self.window = window      # Length of the rolling window in seconds.
        self.calls = []          # Accepted request timestamps.

    # Determine whether a request should be accepted.
    #
    # Any timestamps that fall outside the current rolling window are removed
    # before evaluating the request. If the number of remaining requests is
    # below the configured threshold, the request is accepted and recorded.
    #
    # Args:
    #     timestamp (int | float): Timestamp associated with the request.
    #
    # Returns:
    #     bool:
    #         True if the request is allowed.
    #         False if accepting the request would exceed the configured
    #         request limit.
    def allow(self, timestamp):
        # Discard request records that are older than the active window and
        # therefore no longer contribute to rate-limit calculations.
        self.calls = [
            t for t in self.calls
            if timestamp - t < self.window
        ]

        if len(self.calls) < self.limit:
            self.calls.append(timestamp)
            return True

        return False        # Deny the request because the limit is met.

def retry(func, attempts=3, exceptions=(Exception,)):
    # Execute a callable and automatically retry on recoverable failures.
    #
    # The function is invoked repeatedly until it succeeds or the maximum
    # number of attempts has been reached. Only exceptions matching the

```

```
# supplied exception types trigger a retry; all other exceptions are
# propagated immediately.
#
# Args:
#     func (callable): Operation to execute.
#     attempts (int): Total number of execution attempts, including the
#         initial invocation.
#     exceptions (tuple[type, ...]): Exception types that should be
#         intercepted and retried.
#
# Returns:
#     Any: The value returned by the first successful invocation of func.
#
# Raises:
#     Any exception not listed in exceptions.
#     The final matching exception if all retry attempts fail.
for i in range(attempts):
    try:
        return func()          # Attempt to execute the target operation.

    except exceptions:
        # Once all configured attempts have been used, re-raise the most
        # recent exception so the caller can handle the failure.
        if i == attempts - 1:
            raise

# Defensive fallback. Execution should never reach this point because
# the function either returns successfully or raises an exception.
return None
```

